

Line-by-Line Code Analysis: billing.py

Detailed walk-through of the UPI Payment Validation & Top-Up Processing Engine

PART 1: IMPORTS, FILE PATHS & STATUS CONSTANTS

Line	Source Code Snippet	Detailed Explanation
1-6	<pre>import os import json import re import time import math from typing import Tuple, List, Dict, Any</pre>	Imports core libraries: `os` for path management, `json` for persistence, `re` for UTR regex validation, `time` for timestamps, `math` for numeric checks, and type hints.
8-10	<pre>DATA_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "data")) PAYMENTS_FILE = os.path.join(DATA_DIR, "payments.json")</pre>	Calculates absolute path to the global `data/` directory and sets the `payments.json` storage file path.
12-18	<pre>STATUS_PENDING = "PENDING" STATUS_APPROVED = "APPROVED" STATUS_PARTIAL = "PARTIAL" STATUS_TOPUP_DONE = "TOPUP_DONE" STATUS_FLAGGED = "FLAGGED" STATUS_INVALID_UTR = "INVALID_UTR"</pre>	Defines global string constants representing the status lifecycle of payment verification records.

PART 2: FILE IO & UTR FORMAT VALIDATION

Line	Source Code Snippet	Detailed Explanation
21-23	<pre>def _ensure_data_dir(): """Ensures that the data directory exists.""" os.makedirs(DATA_DIR, exist_ok=True)</pre>	Helper function creating the `data/` directory recursively if it does not already exist.
26-35	<pre>def load_payments() -> List[Dict[str, Any]]: _ensure_data_dir() if not os.path.exists(PAYMENTS_FILE): return [] try:</pre>	Reads and parses payment records from JSON file. Returns an empty list if file is missing or corrupted.

PART 3: PAYMENT & TOP-UP SUBMISSION PROCESSING

Line	Source Code Snippet	Detailed Explanation
54-85	<pre>def submit_payment(plan, required_amount, amount_paid, evals, utr): utr_clean = utr.strip() if not validate_utr_format(utr_clean): ... if not isinstance(amount_paid, (int, float)) or ...: ...</pre>	Validates format & numerical sanity of new payments. Checks for duplicate UTRs across all records.
87-108	<pre> remaining = max(0.0, required_amount - amount_paid) if amount_paid >= required_amount: status = STATUS_PENDING else: status = STATUS_PARTIAL new_record = { ... } payments.append(new_record); save_payments(payments)</pre>	Calculates remaining balance, determines if payment is complete or partial, creates record object, and persists it.
111-163	<pre>def submit_topup(topup_utr, parent_utr, topup_amount): ... pmt["amount_paid"] += topup_amount new_remaining = max(0.0, pmt["required_amount"] - pmt["amount_paid"]) pmt["remaining_balance"] = new_remaining</pre>	Processes top-up proofs for partial payments, ensures top-up UTR uniqueness, and updates balance totals.

PART 4: ADMIN CONTROLS & QUERY UTILITIES

Line	Source Code Snippet	Detailed Explanation
166-174	<pre>def approve_payment(utr: str) -> bool: payments = load_payments() for pmt in payments: if pmt.get("utr") == utr: pmt["status"] = STATUS_APPROVED pmt["remaining_balance"] = 0.0 save_payments(payments) return True</pre>	Admin function to set record status to `APPROVED` and clear remaining balance to zero.